

Chapter 13: Systems Programming (Process Management) and Other Miscellaneous Topics

Instructor: Dr. Md Mahfuzur Rahman
Department of Computer Science, GSU

Zombie Processes

- Zombie processes do not take up system resources
 - But do use up an entry in the system's fixed-size process table
 - Too many zombie processes is a problem

Redirecting file descriptor - dup2()

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
    int fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC,
0644);
    dup2(fd, STDOUT_FILENO); // redirect stdout to file
    close(fd);

    printf("Let's see where this line goes\n");

    return 0;
}
```

stdin Redirection

prog.c

```
#include <stdio.h>
int main() {
    char name[50];
    scanf("%s", name);           // reads from stdin (keyboard)
    printf("%s\n", name);        // writes to stdout (screen)
    return 0;
}
```

Run the program:

```
> gcc prog.c -o prog
> cat file.txt | ./prog
```

OR:

```
> ./prog < file.txt
Output: 235
```

file.txt

235 Mr. X!

stdin redirection using dup2

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
int main() {
    int fd = open("file.txt", O_RDONLY);
    if (fd < 0) {
        perror("open error");
        return 1;
    }
    dup2(fd, STDIN_FILENO); // redirect stdin to file
    close(fd);              // original fd no longer needed
    char buffer[100];
    // This now reads from input.txt instead of keyboard
    fgets(buffer, sizeof(buffer), stdin);
    printf("We just read this from the file: %s", buffer);

    return 0;
}
```

Pipe mechanism

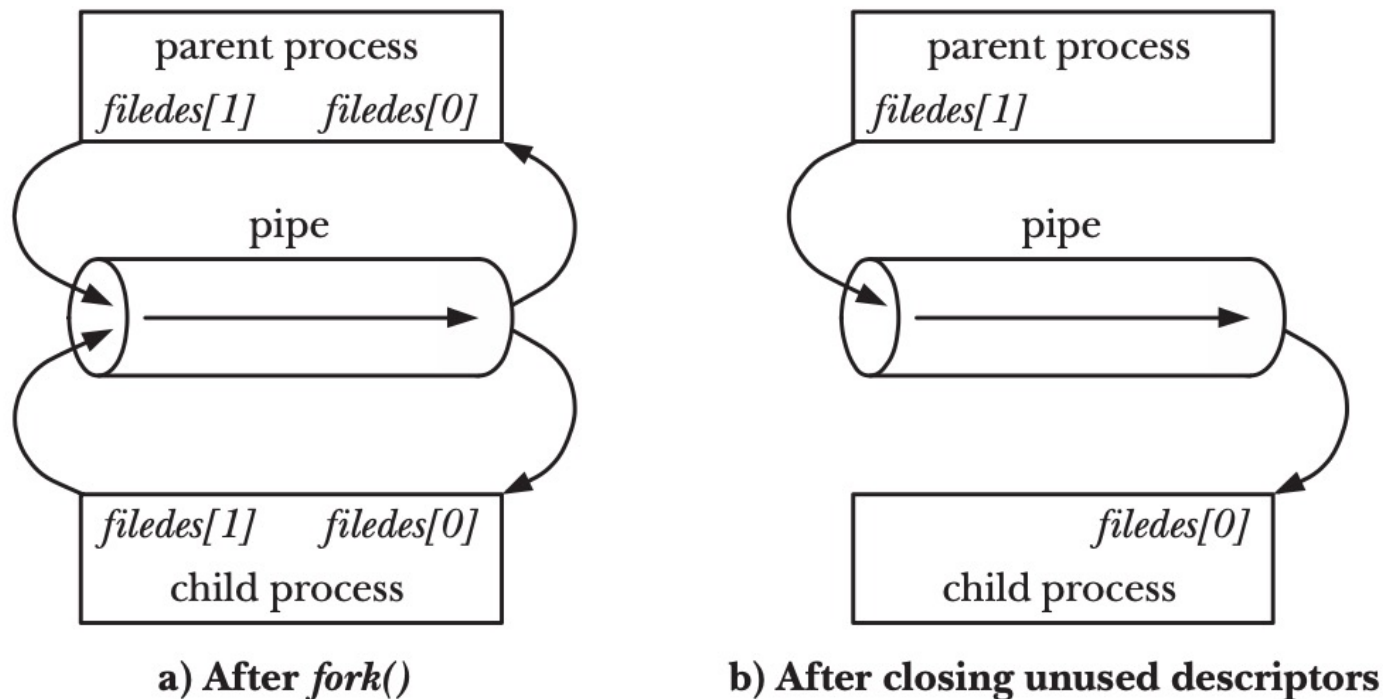


Figure 44-3: Setting up a pipe to transfer data from a parent to a child

Case 1: The reading process tries to read from a pipe, and a write end is open in another process.

- The reader will **block (wait)** until some data is written into the pipe.

Case 2: The reading process tries to read, but all write ends are closed

- In this case, it returns **0** immediately — this indicates **End of File (EOF)**.

Pipe mechanism (contd.)

Case 1:

The writing process writes into a pipe, and the read end is still open (by another process):

- The writing process **writes data into the pipe's buffer**.
- If the pipe buffer has **space**, the write proceeds normally.
- If the pipe buffer is **full**, the write() **blocks** (waits) until the reader reads and frees up space.

Case 2:

The writing process tries to write, but no read end of the pipe is open (i.e., all read descriptors are closed):

- There's **no one to read the data**. The pipe is considered **broken** from the writer's point of view.
- The write() **fails** with a return value of -1, and errno is set to EPIPE.
- Additionally, the system sends a **SIGPIPE signal** to the writing process, which by default **terminates** it.

Question?

Q. A parent process creates a pipe. The parent process forked a child, so the child inherited the pipe's file descriptors. The parent set the read end, and the child set the write end of the pipe. As we don't know the order of execution of the parent and the child processes, how does system ensure the both write end and the read end to be open expectedly? I mean why are we supposed to see normal communication every time we run the code.

Ans: **Fork Ensures Both Ends Exist Initially**

- Right after fork(), both processes still have **both ends** of the pipe open.
- That means from the system's point of view, **at least one write end, and one read end remain open**, regardless of which process gets scheduled first.

Pipe mechanism: `ls -l | wc -l`

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int fd[2]; // fd[0] = read end, fd[1] = write end
    pid_t pid;

    if (pipe(fd) == -1) {
        exit(EXIT_FAILURE);
    }
    pid = fork();
    if (pid == -1) {
        exit(EXIT_FAILURE);
    }
    if (pid == 0) {
        // Child process: executes "ls -l"
        close(fd[0]); // Close unused read end

        // redirect stdout to the write end of the pipe
        dup2(fd[1], STDOUT_FILENO);
        // Close original write end;
        close(fd[1]);

        execlp("ls", "ls", "-l", NULL);
        perror("execlp ls");
        exit(EXIT_FAILURE);
    }
    else {
        // Parent process: executes "wc -l"
        close(fd[1]); // Close unused write end
        // Redirect stdin to read end of pipe
        dup2(fd[0], STDIN_FILENO);
        close(fd[0]); // Close original read end

        execlp("wc", "wc", "-l", NULL);
        perror("execlp wc");
        exit(EXIT_FAILURE);
    }
    return 0;
}
```

Miscellaneous topics: *extern* variable in C

```
/ *extern_example.c
===== */
#include<stdio.h>
// extern is optional for functions
extern int add(int a, int b);
extern int x; // extern does not allocate memory
int main()
{
    printf("The sum = %d\n", add(5, x));
    return 0;
}
```

```
/* extern_file1.c */
int add(int a, int b)
{
    return a + b;
}
int x = 21;
```

Compilation: gcc extern_file1.c extern_example.c -o extern_prog
Run: ./extern_prog



Segmentation Fault – GDB Example

seg.c

```
#include <stdio.h>
#include <stdlib.h>
void bar() {
    int *p = NULL;
    // p = malloc(sizeof(int));
    *p = 12; // expect a seg fault
}
void foo() {
    bar();
}
int main() {
    foo();
    return 0;
}
```

GNU Debugger (GDB) usage

```
[mrahman21@gsuad.gsu.edu@snowball ~]$ gcc -g seg.c -o seg
[mrahman21@gsuad.gsu.edu@snowball ~]$ ./seg
Segmentation fault (core dumped)
[mrahman21@gsuad.gsu.edu@snowball ~]$ gdb ./seg -tui
(gdb) start
(gdb) run
(gdb) backtrace
#0 0x00000000004004fd in bar () at seg.c:7
#1 0x0000000000400513 in foo () at seg.c:11
#2 0x0000000000400523 in main () at seg.c:16
(gdb) q
```

Let's say we have fixed it:

```
(gdb) start
(gdb) s (step into)
(gdb) n
....
(gdb) c
```

If you are interested, read [the GDB manual](#)

Multifile Programs

functions.h

```
const char* print_message()  
{  
    return "Welcome to GSU";  
}
```

main.c

```
int main() {  
    hello();  
    return 0;  
}
```

hello.c

```
#include <stdio.h>  
#include "functions.h"  
void hello()  
{  
    printf("%s\n", print_message());  
}
```

How to run:

Option 1: individually compile each of the c files and then build the final program

Option 2: Compile all the files and build at the same time.

Option 3: Create a makefile, save it as **Makefile** or **makefile** (see the next slide)

- Now run using:

- make final (first)
- make run (second)

Option 3 will be very useful when you will need to deal with big projects. (Compiling and linking will be easier)

Makefile – easy way of managing big project

```
CFLAGS = -Wno-implicit-declaration

all: final

final: main.o hello.o
    @echo "Linking and producing the final executable..."
    gcc $(CFLAGS) main.o hello.o -o final
    chmod +x final

main.o: main.c
    @echo "compiling main.c"
    gcc $(CFLAGS) -c main.c

hello.o: hello.c functions.h
    @echo "Compiling hello.c"
    gcc $(CFLAGS) -c hello.c

clean:
    @echo "Removing unnecessary files..."
    rm main.o hello.o

run:
    ./final
```